Ders 5 Genişletilmiş Notu: Huffman Kodlama: Varyans, Başlık Bilgisi ve Dizi ile Kod Üretimi; Dinamik ve Canonical Huffman; MNP5 ve MNP7

Ders 5 Genişletilmiş Notu: Huffman Kodlama: Varyans, Başlık Bilgisi ve Dizi ile Kod Üretimi; Dinamik ve Canonical Huffman; MNP5 ve MNP7

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Shannon-Fano Tekrarı ve Huffman Kodlamaya Dönüş
- 2. Aynı Olasılıklarla İki Farklı Huffman Ağacı
- 3. Kod Uzunluklarının Varyansı
- 4. Sembol Başına Bit ve Ortalama Uzunluğun Kısa Yoldan Hesabı
- 5. Sıklık Değerleriyle Huffman: Genişletilmiş Dizi
- 6. Diziden Ağacın Kurulması ve Kodların Okunması
- 7. Başlık (Header) Bilgisi ve Ağacın Gönderilmesi
- 8. Genişletilmiş Diziden Doğrudan Kod Üretme
- 9. Statik, Yarı Dinamik ve Dinamik Huffman
- 10. Huffman Algoritmasının Farklı Bir Versiyonu
- 11. Canonical Huffman Kodlama
- 12. MNP5
- 13. MNP7
- Örnek Sorular

1. Shannon-Fano Tekrarı ve Huffman Kodlamaya Dönüş

Hoca derse, geçen hafta en son Golomb kodunu ve Shannon-Fano'yu anlattığını hatırlatarak başladı ve Shannon-Fano'yu sorularla tekrar etti. Shannon-Fano sembollerin istatistiklerine (olasılıklarına) dayalı bir yöntemdir. Olasılıklar 2'nin ters kuvvetleri ($1/2, 1/4, 1/8, \ldots$) biçimindeyse alınabilecek en yüksek başarıyı verir. Semboller olasılığa göre sıralanır, toplam olasılıkları eşit ya da birbirine çok yakın olacak biçimde ikiye bölünür, her parçada tek bir sembol kalana kadar bölmeye devam edilir ve her bölmede bir tarafa 1, diğer tarafa 0 verilir.

Huffman kodlama Shannon-Fano'nun üzerine geliştirilmiş bir yöntemdir ve ona çok benzer; fark ağacın kurulma biçimindedir. Shannon-Fano'da olasılıklar ortalanacak biçimde bölünür, Huffman'da ise olasılık değerleri üzerinde toplama işlemleri yapılarak ağaç kurulur. Huffman tek başına kullanılabileceği gibi başka yöntemlerin içinde ara bir adım olarak da kullanılabilir.

Slayttaki maddeler şöyledir (ekrandan, 00:10:00):

- Veri sıkıştırmada tek başına veya bir başka yöntemin içerisinde yer alarak kullanılan bir algoritmadır.
- Shannon-Fano'ya benzer, ancak kodları ters olarak yukarıdan aşağıya doğru oluşturur.
- Huffman algoritmasında kodlama ağacı oluşturulurken iki farklı yol izlenir. Sembollerin kod değerleri farklı olsa da entropi aynı çıkar.

Hoca önce Huffman'ı hiç anlatmadığını sandı; öğrencilerden gelen cevaplarla geçen dersin sonunda Huffman'a giriş yapıldığı, ama ayrıntısının kaldığı netleşti. Hoca girişi kısaca tekrar ederek konuya devam etti.

2. Aynı Olasılıklarla İki Farklı Huffman Ağacı

Olasılıklar $a_1 = 0{,}4$, $a_2 = 0{,}2$, $a_3 = 0{,}2$, $a_4 = 0{,}1$, $a_5 = 0{,}1$'dir (ekrandan, 00:15:40). Huffman şöyle çalışır: olasılık değerleri küçükten büyüğe sıralanır, ilk iki değer toplanır ve toplam sıralamayı bozmayacak bir yere yerleştirilir. Sıralama `0,1 0,1 0,2 0,2 0,4` iken $0{,}1 + 0{,}1 = 0{,}2$ çıkar. Elde zaten iki tane 0,2 vardır; yeni 0,2 bunların önüne de arkasına da konabilir, iki yer de sıralamayı bozmaz. Bu seçim iki farklı ağaç verir.

2.1 Birinci Yol: Toplamı Eşit Değerlerin Önüne Koymak

Hoca tahtada mavi kalemle gösterdi. $0{,}1 + 0{,}1 = 0{,}2$ eşit değerlerin önüne konur. Sonra $0{,}2 + 0{,}2 = 0{,}4$ çıkar; elde zaten bir 0,4 vardır ve algoritma hep öne koyduğu için yeni 0,4 de öne konur. Sıradaki ikisi toplanınca $0{,}6$, en sonda $0{,}6 + 0{,}4 = 1$ olur. Bu ağaçta kodlar `0, 10, 111, 1101, 1100` çıkar ve ortalama mesaj uzunluğu:

$$S = 0{,}4 \cdot 1 + 0{,}2 \cdot 2 + 0{,}2 \cdot 3 + 0{,}1 \cdot 4 + 0{,}1 \cdot 4 = 2{,}2 \text{ bit/sembol}$$

2.2 İkinci Yol: Toplamı Eşit Değerlerin Arkasına Koymak

Hoca kalem rengini değiştirip aynı işlemi tekrarladı. $0{,}1 + 0{,}1 = 0{,}2$ bu kez eşit değerlerin sonuna konur. $0{,}2 + 0{,}2 = 0{,}4$ de sona konur. Sonra kalan $0{,}2$ ile $0{,}4$ toplanır, $0{,}6$ olur; en sonda $0{,}4 + 0{,}6 = 1$. Kodlar `11, 01, 00, 101, 100` çıkar (ekrandan, 00:20:00):

$$S = 0{,}4 \cdot 2 + 0{,}2 \cdot 2 + 0{,}2 \cdot 2 + 0{,}1 \cdot 3 + 0{,}1 \cdot 3 = 2{,}2 \text{ bit/sembol}$$

> **Ders sonrası not:** Hoca ikinci yolu anlatırken bir yerde toplamlar için "çarptım" dedi ("0,2 ile 0,2'yi çarptım 0,4"); doğrusu toplamadır: $0{,}2 + 0{,}2 = 0{,}4$, $0{,}2 + 0{,}4 = 0{,}6$, $0{,}6 + 0{,}4 = 1$.

2.3 İki Ağacın Karşılaştırılması

Olasılıklar aynı olduğu için iki sistemin entropisi aynıdır. Ortalama mesaj uzunlukları da aynı (2,2) çıktı; dolayısıyla entropiyle ortalama uzunluk arasındaki fark olan artıklık (redundancy) da aynıdır. Entropi derste sayısal olarak hesaplanmadı; python ile $H \approx 2{,}12$ bit, artıklık $R \approx 0{,}08$ bit bulunur.

Öyleyse neden iki ağaçla uğraşılıyor? Kodlara bakınca fark görülür: birinci ağaçta 1 bitlik bir kod vardır, ikincisinde yoktur; birincide en uzun kod 4 bit, ikincide 3 bittir. Entropi, ortalama uzunluk ve artıklık aynı olduğu halde kodların bit uzunluklarının farklı dağılması bir şey anlatır. Hoca bunu şöyle bağladı: veri hiçbir iletim hattı kullanmadan kendi makinemde sıkıştırılıp açılıyorsa hangi ağacın kullanıldığının önemi yoktur. Ama hatlardan kaynaklı beklemeleri önlemek için bit uzunlukları arasındaki varyansı küçük olan ağaç tercih edilmelidir.

3. Kod Uzunluklarının Varyansı

3.1 Varyans Sezgisi

Varyans, ortalamaya göre değişimdir. Hoca iki dizi verdi. `1 1 2 1 1` dizisinde değerler birbirine çok yakındır; her birinin ortalamadan farkı küçüktür, varyans küçüktür. `3 1 7 4` dizisinde ise değerler arasındaki farklar büyüktür, ortalamadan uzaklıklar da büyüktür, varyans büyüktür.

Huffman ağaçlarına dönersek: birinci ağaçta kod uzunlukları `1, 2, 3, 4, 4`, ikincide `2, 2, 2, 3, 3`'tür. İkincide uzunluklar birbirine daha yakın olduğu için varyans daha küçük olacaktır. Hoca gerçek uygulamalarda çok daha fazla sembol olacağını, kodların 17 ya da 25 bit uzunluğunda olabileceğini; varyansın küçüklüğünün kodların bit sayılarının birbirine yakınlığı demek olduğunu vurguladı.

> **Ders sonrası not:** Hoca varyansı "dizinin ortalamasıyla her bir değer arasındaki farkların karelerinin karekökü" diye tanımladı. Farkların karelerinin ortalamasının karekökü standart sapmadır; varyans, farkların karelerinin ortalamasıdır. Slayttaki hesap (aşağıda) da karekök almadan, olasılıkla ağırlıklandırılmış kareler toplamı biçimindedir.

3.2 Varyans Hesabı

Slayttaki hesap (ekrandan, 00:30:00), her kod uzunluğunun ortalama mesaj uzunluğundan (2,2) farkının karesini sembolün olasılığıyla çarpıp toplar:

$$\sigma_1^2 = 0{,}4(1-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}2(3-2{,}2)^2 + 0{,}1(4-2{,}2)^2 + 0{,}1(4-2{,}2)^2 = 1{,}36$$

$$\sigma_2^2 = 0{,}4(2-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}1(3-2{,}2)^2 + 0{,}1(3-2{,}2)^2 = 0{,}16$$

İki sonuç da python ile doğrulandı.

3.3 Varyans Neden Önemli?

Hoca bunu iki şehir arasında veri gönderme örneğiyle anlattı. A şehrinde veri sıkıştırılıp B şehrine (örnekte Ankara'ya) gönderiliyor. Dinamik Huffman'da karşı taraf gelen veriyi anında açmak zorundadır, çünkü iki taraf aynı anda aynı işlemleri yapar. Kodların bazıları az bitli, bazıları çok bitliyse alıcıda bekleme oluşur: arkadan bir bitlik bir kod gelse bile önceki dört bitlik kod henüz çözülmediği için onu işleyemez. Bu yüzden alıcıda bir tampon bellek (buffer) ayrılır; gelenler oraya atılır, B tarafı işini bitirdikçe sıradakini alır. Varyans büyükse bekleme çok olur ve kayıp olmaması için buffer büyük ayrılmalıdır. Varyans küçükse gönderme ile açma hemen hemen aynı sürede olur ve buffer küçük tutulabilir. Belleği sınırlı bir sistemde büyük bir alan ayırmamak için varyansı küçük ağaç seçilir.

Yarı adaptif (semi-adaptive) ya da statik bir Huffman kullanılıyorsa iki ağaçtan hangisinin kullanıldığı fark etmez, sıkıştırma performansı aynıdır. Adaptif Huffman'da ise gönderilen veri karşıda anında açılmaya çalışıldığı için kod çözme sürelerinin birbirine yakın olması, yani varyansın küçük olması gerekir.

Slayttaki maddeler (ekrandan, 00:30:00):

- Kodlayıcı sıkıştırılmış veriyi bir dosyaya yazacaksa, kodun varyansının küçük veya büyük olmasının bir önemi yoktur.
- Kodlayıcı sıkıştırılmış veriyi iletişim hatlarından gönderecekse, küçük varyanslı kod seçilmelidir.
- İletişim hattı üzerinden bilginin gönderilme hızı ile kodlayıcının kodu üretme hızı aynı değilse beklemeyi önlemek için buffer alanı kullanılır.
- Kodun varyansı büyükse buffer alanı büyük seçilmelidir. Varyansın sıfıra yakın olduğu durumlarda buffer alanı küçük seçilebilir.

  Hocanın vurgusu: Varyansın önemi yalnızca bir veri sıkıştırılarak bir iletim hattı üzerinden gönderilirken ortaya çıkar.

4. Sembol Başına Bit ve Ortalama Uzunluğun Kısa Yoldan Hesabı

4.1 Bir Sembol Kaç Bitle Kodlanır?

Slaytta şu kural yazar (ekrandan, 00:30:00): a_i sembolünün kod uzunluğu $\lceil -\log_2 P_i \rceil$ ile eşit veya daha küçüktür. Örnek: $P = 0{,}4$ için $\lceil -\log_2 0{,}4 \rceil = 2$ bit; sembol 2 ya da 1 bitle kodlanır. Olasılık 0 ile 1 arasında olduğu için logaritması hep negatiftir; önüne eksi konmasının sebebi sonucu pozitife çevirmektir.

Hoca derste tavan işaretinin yönünden emin olamadı ve sonradan bakacağını söyledi. Aradan sonra bir öğrenci $\log_2 0{,}4$'ün yaklaşık $-1{,}32$ olduğunu, dolayısıyla yazılan 2'nin doğru olduğunu söyledi. Gerçekten $-\log_2 0{,}4 \approx 1{,}32$ ve $\lceil 1{,}32 \rceil = 2$'dir (python ile doğrulandı). Tavan işareti (yukarı yuvarlama) $2{,}1$ için 3, taban işareti (aşağı yuvarlama) 2 verir.

> **Ek bilgi (derste anlatılmadı):** Slayttaki "eşit veya daha küçüktür" ifadesi bu örnek için doğrudur (a_1 1 ya da 2 bit aldı). Genel olarak Huffman'da çok düşük olasılıklı bir sembolün kodu $\lceil -\log_2 P_i \rceil$'den uzun olabilir; garanti edilen, ortalama uzunluğun $H \le S < H + 1$ aralığında kalmasıdır.

4.2 Ortalama Mesaj Uzunluğunun Kısa Yolu

S hesaplanırken her olasılık, o olasılıklı sembolü kodlamak için gereken bit sayısıyla çarpılır. Bit sayısı ağaçta kökten o sembole kadar kaç dal (büküm) varsa odur. Birinci ağaçta 0,4 bir bitle, 0,2 iki bitle, diğer 0,2 üç bitle, iki 0,1 de dört bitle kodlandığı için $S = 2{,}2$ çıkmıştı.

Slayt bunun daha kolay bir yolunu da verir: ağaçta toplanarak oluşan ara değerlerin (iç düğümlerin) toplamı ortalama mesaj uzunluğunu verir. Birinci ağaçta ara toplamlar $0{,}2$, $0{,}4$, $0{,}6$ ve en dıştaki $1{,}0$'dır:

$$0{,}2 + 0{,}4 + 0{,}6 + 1{,}0 = 2{,}2$$

Bu, S formülüyle bulunan değerin aynısıdır (python ile doğrulandı).

5. Sıklık Değerleriyle Huffman: Genişletilmiş Dizi

5.1 Örnek

Slayttaki örnek (ekrandan, 00:33:30): elimizde `a, e, k, l, m` ve `z` sembollerinden sırasıyla 25, 14, 8, 7, 4 ve 2 adet olsun; her bir sembol için Huffman kod değerini bulalım. Burada olasılık değil kullanım sıklıkları verilmiştir. Olasılık istenirse toplam 60 olduğu için $P(a) = 25/60$, $P(e) = 14/60$, $P(k) = 8/60$ vb. olur. Huffman olasılıklarla da sıklık değerleriyle de yapılabilir.

5.2 Adım 1 ve Adım 2

Adım 1: sembollerin kullanım sıklıkları küçükten büyüğe sıralanır: `2 4 7 8 14 25`. Hoca slaytta değerlerin tesadüfen sondan verildiğini, ama mutlaka küçükten büyüğe sıralanması gerektiğini söyledi.

Adım 2: sıklık değerleri, soldan başlanarak sıradaki iki değerin toplamı sıralamayı bozmayacak biçimde diziye yerleştirilerek genişletilir. Toplamlar negatif işaretle yazılır:

```
başlangıç:   2  4  7  8  14  25
2+4=6    ->  2  4  -6  7  8  14  25
6+7=13   ->  2  4  -6  7  8  -13  14  25
8+13=21  ->  2  4  -6  7  8  -13  14  -21  25
14+21=35 ->  2  4  -6  7  8  -13  14  -21  25  -35
25+35=60 ->  2  4  -6  7  8  -13  14  -21  25  -35  -60
```

Neden eksi işaret? Hoca kırmızıyla yazdığı ara toplamları bir insanın renkten ayırt edebileceğini, ama bilgisayara bunu anlatmanın bir yolu gerektiğini söyledi. Toplam diziye negatif işaretle konursa algoritma bunun gerçek bir sıklık (ya da olasılık) değil, iki değerin toplamından oluşan bir ara değer olduğunu bilir. Toplarken işaret dikkate alınmaz: -6 ile 7 toplanırken 1 değil 13 bulunur. İşaretin tek anlamı, değerin ara toplam olduğunu göstermesidir.

> **Ders sonrası not:** Hoca -21'in yerini söylerken "14 ile 15'in arasına" dedi; dizide 15 yoktur, -21 14 ile 25'in arasına yerleşir.

5.3 Dizinin Boyu

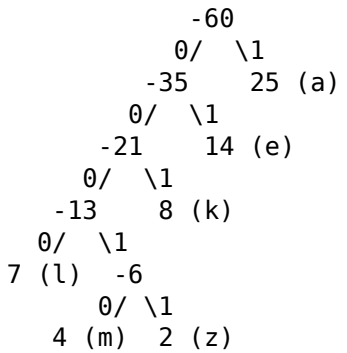
n tane sembol varsa genişletilmiş dizi $2n-1$ elemanlı olur. Örnekte $n=6$, dizi $2\cdot 6-1=11$ elemanlıdır; hoca elemanları sayarak doğruladı. Hoca, dizi $2n-2$'ye kadar genişletilseydi en sondaki toplamın (60) olmayacağını ve köke bir değer koymaya gerek kalmadan dallardan başlanacağını söyledi.

6. Diziden Ağacın Kurulması ve Kodların Okunması

6.1 Adım 3: Ağacın Kurulması

Slayttaki kural (ekrandan, 00:40:00): en sondaki eleman Huffman kodlama ağacının kök değeri olur, diğer elemanlar da ağaca belli bir kuralla yerleştirilir. Bir sonraki ($2n-2$. konumdaki) değer ağacın sol dalına, ($2n-3$. konumdaki) değer de sağ dalına yerleşir.

Kök -60'tır. Sondan geriye doğru ikişer değer okunur: ilk okunan -35 sol dala, ikinci okunan 25 sağ dala konur. Doğruluk kontrolü: $35+25=60$. 25 pozitif olduğu için gerçek bir sıklıktır, yani ağacın yaprağıdır; slaytta yuvarlak içine alınmıştır. -35 ise bir şeylerin toplamıdır, ara düğümdür. Sıradakiler: -21 sola, 14 sağa ($21+14=35$). Sonra -13 sola, 8 sağa. Sonra 7 sola, -6 sağa ($7+6=13$). En son 4 sola, 2 sağa. Ağaç tamamlanır:



> **Hocanın vurgusu:** Ağaç her zaman aynı kuralla kurulmalıdır. İlk okunanı hep sola koyuyorsanız hep öyle yapın. Bazen sola bazen sağa koyarsanız yanlış bir ağaç oluşur ve çıkan kodlar prefix olmaz.

6.2 Adım 4-5: Kodların Okunması

Dallara etiket verilirken de aynı kural geçerlidir: bütün sol dallara 0, bütün sağ dallara 1 verilir. Slayttaki Adım 5 (ekrandan, 00:50:00): Huffman kodlama ağacında kodlar aşağıdan yukarı doğru oluşturulur, yukarıdan aşağı doğru okunur. Kökten yaprağa giden yol üzerindeki dal etiketleri sembolün kodudur:

sembol	sıklık	kod
a	25	1
e	14	01
k	8	001
l	7	0000
m	4	00010
z	2	00011

Kodlar aşağıdan yukarı okunsaydı `a = 1`, `e = 10`, `k = 100`, `l = 0000`, `m = 01000`, `z = 11000` olurdu. Bu kodlar prefix değildir: hepsi 1 ile başlayan birkaç kod vardır; açıcı `10` gördüğünde bunun `e` mi yoksa `k`'nin başı mı olduğunu bilemez.

Derste hesaplanmadı ama bu kodlarla toplam bit sayısı $25\cdot 1+14\cdot 2+8\cdot 3+7\cdot 4+4\cdot 5+2\cdot 5=135$, sembol başına ortalama $135/60=2{,}25$ bittir. Bölüm 4.2'deki kısa yol burada da çalışır: ara toplamlar $6+13+21+35+60=135$ (python ile doğrulandı).

7. Başlık (Header) Bilgisi ve Ağacın Gönderilmesi

7.1 Header Neden Gerekir?

Sıkıştırılmış dosya karşı tarafa gönderilecekse, açıcının `a, e, k, l, m, z` sembollerinin kodlarını bilmesi gerekir; yoksa açamaz. Bu yüzden sıkıştırılmış dosyanın başına bir başlık (header) eklenir. Hoca bunu ödevde bitmap dosyası açılırken kullanılan header'a benzetti: decompress algoritması da bu header'ı kullanır.

Header'da şunlar olabilir: kaç sembol kodlandığı, semboller için kaç bit harcandığı, sembollerin kodları; ya da semboller ve sıklık değerleri. Örneğin birinci bayttan alfabenin kaç karakterden oluştuğu (6) okunur, sonra her sembol için bir bayt sembol, bir bayt sıklık okunur; bu altı kez tekrarlanır.

Sorun şudur: bir sembolün sıklığı 3, bir başkasınınki 256, bir başkasınınki 4096 olabilir. Bunlar farklı sayıda bitle yazılabilir; ama her biri için bayt bayt yer ayrılır ve bir sıklık bir bayta sığmıyorsa iki bayt ayrılırsa, açıcı okuduğu sıklığın bir bayta mı iki bayta mı yazıldığını bilemez. O zaman hepsi için iki bayt ayırmak gerekir. Bitler kullanılsa bu kez her değerin kaç bitle yazıldığı bilgisinin de bir yerde tutulması gerekir. Böylece header'a ek bilgiler girer ve header bazen sıkıştırılmış verinin boyutuna kadar büyür; hoca bu durumda "veriyi sıkıştırmasan daha iyi" dedi.

Bir başka yol, sembolleri ve sıklıkları gönderip ağacı karşı tarafta yeniden kurdurmaktır. Hoca, zaten kurulmuş bir ağacın neden karşı tarafta yeniden kurulduğunu sordu: ağaç olduğu gibi gönderilebilir. Ama bir ağacın resmi gönderilmeyeceğine göre bunun bir veri yapısı yolu gerekir. Hoca burada veri yapıları ve ayrık matematik derslerindeki ağaç okuma notasyonlarını hatırlattı.

7.2 Left-Node-Right Notasyonu

Slayttaki soru (ekrandan, 01:00:00): başlık içerisinde ağaç bilgisini gönderebilir miyiz? Nasıl? Ağacımızı Left Node Right notasyonuna göre okuyalım.

Basit bir örnek: kökü C, sol yaprağı A, sağ yaprağı B olan bir ağacın Left-Node-Right okuması `A C B`'dir. Huffman ağaçlarında ara düğümlerde sembol olmadığı için düğüme çıkıldığında okunacak bir veri yoktur; okunan şey dalların etiketleridir, yapraklar da sembollerdir.

Örnek ağaç birlikte okunur (ekrandan, 01:00:00 ve 01:10:00):

- Kökten sola inilir: `0`. Yeni alt ağaçta yine sola: `0`. Yine sola: `0`. Yine sola: `0`. Artık gidecek yer yoktur; yaprak `l` alınır.
- Bu alt ağacın sol kolu bitti; düğüme çıkılır, sağa inilir: `1`. Buradaki alt ağacın soluna inilir: `0`, yaprak `m`. Düğüme çıkılır, sağa: `1`, yaprak `z`.
- Bu alt ağaç bitti; bir üst düğümden sağa: `1`, yaprak `k`.
- Bir üst düğümden sağa: `1`, yaprak `e`.
- Kökten sağa: `1`, yaprak `a`.

Sonuç bit dizisi `0000101111`, yaprak sırası `l m z k e a`'dır. Header'a önce bir bayta kaç bit okunacağı (10) yazılır, sonra bu bit dizisi, arkasından da semboller eklenir; sembollerin her biri bir bayttır. Hoca header'ın başında bunlardan önce başka bilgiler de olabileceğini, bunun bitiminde sıkıştırılmış verinin başladığını gösterdi.

Hoca ikinci bir rastgele ağaç çizerek okumayı birlikte tekrarladı: kökün sol alt ağacının solunda yaprak A, sağında iki yapraklı (B ve C) bir alt ağaç; kökün sağ alt ağacının solunda yaprak E, sağında iki yapraklı (F ve G) bir alt ağaç vardır. Okuma: `0` (kökten sola), `0` → A, `1`, `0` → B, `1` → C, `1` (kökten sağa), `0` → E, `1`, `0` → F, `1` → G. Hoca bu bilginin header'a konursa karşı tarafın aynı ağacı kuracağını söyledi.

7.3 Karşı Tarafta Ağacın Yeniden Kurulması

Açıcı, bit dizisinin Left-Node-Right notasyonuna göre oluşturulduğunu bilir ve ağacı buna göre yeniden kurar (ekrandan, 01:10:00):

- Kökte beklenir. `0` gelir: sol dallara 0 verildiği bilindiği için bir sol kol açılır. Bir `0` daha: bir sol kol daha. Bir `0` daha, bir `0` daha: dört sol kol.

- Sonra `1` gelir. Sol taraf bitmiş demektir; bulunulan yer yapraktır, ilk sembol `l` buraya yazılır. Düğüme çıkılıp sağa inilir.
- `0` gelir: sola inilir. `1` gelir: buradaki sol kol bitmiştir, sıradaki sembol `m` yazılır; düğüme çıkılıp sağa inilir.
- Şimdi ilginç bir durum vardır: sıradaki bit yine `1`'dir, `0` yoktur, yani sola gidilmiyor. Sağa da gidilemez, çünkü Huffman algoritması sol kol olmadan sağ kolu oluşturmaz; sol dolmadan sağ kol çıkmaz. Bunun anlamı, bu düğümün bir yaprak olduğudur: sıradaki sembol `z` buraya yazılır ve bir üst seviyeye çıkılır. Sağ dalı henüz işlenmemiş ilk düğüme kadar çıkılıp onun sağına inilir; bu `1` o dalı temsil eder.
- Yine `1` gelir; burada da sol olmadığı için yaprak vardır, `k` yazılır; yukarı çıkılıp sağa inilir. Yine `1`: `e` yazılır; kökün sağına inilir.
- Header'daki bayt kaç bit okunacağını söylüyordu; o kadar bit okundu, ağaçla ilgili okunacak bit kalmadı. Son sembol `a` son yaprağa yazılır.

Artık hangi sembolün hangi koda sahip olduğu bu ağaçtan çıkarılır. Sonra sıkıştırılmış veri sırasıyla okunur: `0` diye bir kod var mı? Yok, bir bit daha. `00`? Yok. `000`? Yok. `0000`? Var: `l`. `l` yazılır ve bit bit okumaya devam edilerek bütün dosya açılır.

> **Ek bilgi (derste anlatılmadı):** Bu yöntemin çalışmasının sebebi, Huffman ağacının "tam" bir ikili ağaç olmasıdır: her iç düğümün tam iki çocuğu vardır. Bu yüzden dal etiketleri dizisi ve yaprak sırası, ağacın biçimini tek anlamlı olarak belirler.

Hoca header'larda karşılaşılabilecek başka bilgilere de değindi: birinci baytta sembol sayısı, arkasından semboller, arkasından sıklık değerleri; sıklıklar için örneğin 7 bit kullanıldıysa bunu belirten bir bilgi; ayrıca son baytta özgün veriye ait 8 bitin yalnız birkaçı (örneğin 5'i) bulunabileceği için son baytta kaç bitin okunacağını gösteren bir bayt. Algoritmayı yazan birinin bu son baytı da kontrol etmesi gerekir.

8. Genişletilmiş Diziden Doğrudan Kod Üretme

8.1 Düğüm Adresleri

Ağaç üzerinde sola, sağa gitmeyi bir kodda nasıl gerçekleştireceğiz? Slayttaki soru (ekrandan, 01:20:00): genişletilmiş diziyi kullanarak kodları nasıl oluşturabiliriz? Ağaç üzerindeki her düğüm noktasının adresleri yazılır. Kök düğümün adresi 0 ile başlar; sol taraftaki düğüm kendisinin $\times 2 + 1$'i, sağ taraftaki düğüm $\times 2 + 2$'si olur.

Hoca bunu ağacın bir diziye taşınmasına benzetti. Ağaç tam (full) bir ikili ağaç olsaydı indisler 0, 1, 2, 3, … diye ardışık dizilirdi; Huffman ağaçları hiçbir zaman tam (full binary) olmadığı için indisler ardışık olmaz, ama kural aynıdır: sol $2i + 1$, sağ $2i + 2$. Örnekte: kök 0; sol 1, sağ 2; 1'in çocukları 3 ve 4; 3'ün çocukları 7 ve 8; 7'nin çocukları 15 ve 16; 16'nın çocukları 33 ve 34.

8.2 İndislerin Algoritmayla Bulunması

Slayt (ekrandan, 01:30:00): her negatif sayı bir ara düğüm olduğu için bu düğümden çıkan dallar vardır. Bu sebeple her negatif sayının sahip olduğu indis değerinin $\times 2 + 1$ ve $\times 2 + 2$ değeri peş peşe indis sütununa yazılır. Örneğin -60 bir ara düğümdür, indis değeri 0'dır; $\times 2+1$ ve $\times 2 + 2$ değerleri sırasıyla 1 ve 2 olup arka arkaya yazılır.

Genişletilmiş dizi sondan başlanarak bir değişkene alınır: $-60, -35, 25, -21, 14, -13, 8, 7, -6, 4, 2$. İşlem:

- -60 (indis 0) negatiftir, altında iki dal vardır: 1 ve 2. Gerçekten -35 1 numaralı, 25 2 numaralı indistedir.
- -35 (indis 1) negatiftir: 3 ve 4. -21 3'te, 14 4'tedir.
- 25 pozitiftir, yapraktır; işlem yapılmaz.
- -21 (indis 3): 7 ve 8. -13 7'de, 8 8'dedir.
- 14 pozitif, atlanır. -13 (indis 7): 15 ve 16. 7 15'te, -6 16'dadır.
- 8 ve 7 pozitiftir. -6 (indis 16): 33 ve 34. 4 33'te, 2 34'tedir. 4 ve 2 pozitiftir.

```
dizi   indis  kod
-60    0
```

```
-35     1
25      2       1
-21     3
14      4       01
-13     7
8       8       001
7       15      0000
-6      16
4       33      00010
2       34      00011
```

Sonuçta elde yalnız sıklık değerleri vardı; bunlar genişletildi, soldan başlanarak yerleştirildi ve indisler çıkarılınca hangi değerin ağacın hangi gözünde olduğu bilinir hale geldi. Elde yalnızca iki dizi vardır.

8.3 Kodların Bulunması

Slayt (ekrandan, 01:33:45): Huffman ağacı kodları aşağıdan yukarı doğru oluşturduğundan, kodların prefix sağlaması için yukarıdan aşağıya doğru okunması gerekir. Matriste kod sütunu oluşturulurken de ağaca indisleri yerleştirirken yapılan işlemin tersi yapılır. Dizinin pozitif elemanları ağacın yapraklarına karşılık gelip sembolü temsil ettiğinden, sahip olduğu indis değeriyle işlem yapılır; elde edilen kod değerleri de sağdan sola doğru yazılır:

```
While indis <> 0 do
{
  if indis çift then
    { print "1"
      indis = (indis-2)/2 }
  else
    { print "0"
      indis = (indis-1)/2 }
}
```

Ters işlem şudur: 1'e $0 \times 2 + 1$ ile gelinmişti; tersine 1 çıkarılıp 2'ye bölünür, 0'a dönülür. 4'e ise sağdan, $1 \times 2 + 2$ ile gelinmişti; 2 çıkarılıp 2'ye bölünür, 1'e dönülür. Sağ dallar çift, sol dallar tek indislidir; bu yüzden çift indiste 1, tek indiste 0 yazılır.

Adım adım (ara adımlar hocanın anlatımından):

- 25 (indis 2): çift → `1`; $(2-2)/2 = 0$; döngü biter. Kod `1`. Ağaçta da 25'in kodu 1'dir.
- 14 (indis 4): çift → `1`; $(4-2)/2 = 1$. 1 tek → `0`; $(1-1)/2 = 0$. Bulunan bitler `1, 0`; sağdan sola yazılınca kod `01`.
- 8 (indis 8): çift → `1`; $(8-2)/2 = 3$. 3 tek → `0`; $(3-1)/2 = 1$. 1 tek → `0`; 0. Kod `001`.
- 7 (indis 15): tek → `0`; $14/2 = 7$. 7 tek → `0`; 3. 3 tek → `0`; 1. 1 tek → `0`; 0. Kod `0000`.
- 2 (indis 34): çift → `1`; $32/2 = 16$. 16 çift → `1`; $14/2 = 7$. 7 tek → `0`; $3 \to$ `0`; $1 \to$ `0`; 0. Bitler `1, 1, 0, 0, 0`, sağdan sola `00011`.
- 4 (indis 33): aynı yolla `00010` (ara adım eklendi: 33 tek → `0`, 16; çift → `1`, 7; sonra üç kez `0`).

Bütün kodlar ağaçtan okunanlarla aynıdır (python ile doğrulandı).

Soru: Neden ilk bulunan 1 yazıldı da yeni bulunan onun önüne kondu? **Cevap:** İşlem aşağıdan yukarı doğru, yapraktan köke ilerler. Kodlar aşağıdan yukarı oluşturulur ama yukarıdan aşağı okunduğu için oluşturma sırası tersten yazılır. `e` için bulunan bitler `10` diye yazılsaydı kod prefix olmazdı, çünkü önünde 1 ile başlayan başka bir kod (`a = 1`) vardır.

> **Hocanın vurgusu:** Bu kadar anlatılan şey aslında iki diziden ibarettir; iki çıkarıp ikiye bölerek, bir çıkarıp ikiye bölerek Huffman kodları otomatik elde edilir. Yani Huffman karmaşık bir algoritma değildir. Header'da ağaç gönderilirse algoritmanın header kısmı da kısalır; sıkıştırırken ve açarken kodlaması kolaydır.

8.4 Huffman'ın Başarısı

Hoca başarının verinin içeriğine bağlı olarak değiştiğini, ama genelde doğal dilde yazılmış bir metinde %65'e kadar çıktığını söyledi. %70 çok nadirdir; bunun için içerikte çok tekrar eden olayların geçmesi gerekir. Statik Huffman'la başarı biraz daha düşer, dinamik Huffman'la daha yükselir.

9. Statik, Yarı Dinamik ve Dinamik Huffman

9.1 Üç Tür

Slayttaki tanımlar (ekrandan, 01:40:00):

- **Statik Huffman kod:** Bir derlem içerisinde yer alan tüm sembollerin kullanım sıklıkları çıkarıldıktan sonra her sembol için Huffman kod değerleri oluşturulur. Gelen her dokümanda aynı kod değerleri kullanılır; doküman içerisindeki sembollerin kullanım sıklıkları dikkate alınmaz.
- **Yarı dinamik Huffman kod:** Kodlanacak her doküman üzerinde iki aşamadan oluşan bir işlem yapılır. İlk olarak doküman baştan sona taranarak sembollerin kullanım sıklıkları çıkarılır, ikinci adımda Huffman kod değerleri elde edilir.
- **Dinamik Huffman kod:** Hızın önemli olduğu uygulamalarda kullanılır.

Hoca statik Huffman'ı geçen haftaki hazır prefix kodlara benzetti: dokümanlardan oluşan bir derlem alınır, karakterlerin olasılıkları çıkarılır ve kodlar hazırlanıp size verilir. Sonrası size kalır: yeni metindeki karakterlerin olasılıkları çıkarılır, en yüksek olasılıklıya en kısa kod atanır. Yarı dinamikte hazır bir set yoktur; demin yapıldığı gibi iki geçiş vardır: birinci geçişte veri okunup olasılıklar çıkarılır, Huffman ağacı kurulur ve kodlar çıkarılır; ardından veri bu kodlarla sıkıştırılır. Kodlar metnin içeriğine göre üretildiği için yarı dinamik Huffman statikten daha iyi performans verir.

Üçünde de aynı Huffman ağacı vardır. Birincide kodlar çok önceden hazırlanır, ikincide eldeki veriye göre hazırlanır. Dinamikte yine eldeki veriye göre hazırlanır, ama bir sembolün kodu doküman içinde defalarca değişebilir: sayfanın ortalarında 4 bitle, sonlarına doğru 2 bitle kodlanabilir. Bu yüzden en iyi performansı dinamik Huffman verir. Ancak dinamik Huffman genelde sıkıştırılmış veriyi bir iletim hattında gönderirken tercih edilir; kendi bilgisayarında bir dosyayı sıkıştıran için dinamik kodlamanın bir anlamı yoktur.

9.2 Dinamik Huffman'ın İşleyişi

Slayttaki maddeler (ekrandan, 01:40:00):

- Encoder ve decoder senkronize çalışarak aynı anda kodlama ağacını oluşturur.
- Başlangıçta ağaç boştur. Okunan her yeni sembol ile ağaç yeniden oluşturulur.
- Input stream'den ilk sembol okunur ve kodlanmadan output stream'e yazılır.
- Sembol ağaca eklenir ve kendisine bir kod atanır.
- Daha sonraki adımlarda bu sembol ile karşılaşılırsa, önce sembolün kodu gönderilir, sonra da frekans değeri bir artırılır.
- Decoder'ın bilmesi gereken, gönderilen sembolün kodlanarak mı yoksa kodlanmadan mı gönderildiğidir. Kodlanmamış sembolden önce escape kodu gönderilir.
- Decoder, escape kodundan sonra gelen kodun ASCII koda sahip bir sembol olduğunu anlar.
- Escape kodun frekans değeri her zaman 1'dir ve değişmez. Ancak her yeni sembol ağaca eklendikçe kodu değişir.

Hoca dinamik Huffman'da da varyansı küçük ağacın tercih edilmesi gerektiğini tekrarladı: iki değer toplandığında eşit değer varsa toplam onların arkasına konursa kodların varyansı daha küçük olur. İletim hattında iki taraf senkron çalışır; biri gönderir, öteki hemen çözer; beklemenin az olması için varyans küçük olmalıdır.

Escape örneği (ekrandan, 01:42:40): başlangıçta iki tarafın ağacında yalnız escape kodu vardır ve kodu `0`'dır. Input'ta `b` vardır ve ağaçta yoktur. Gönderici önce `0`'ı gönderir; karşı taraf bunun escape olduğunu, ardından kodlanmamış bir sembol geleceğini anlar. Sonra `b` olduğu gibi (ASCII) gönderilir. İki taraf da aynı algoritmayla `b`'yi ağaca ekler; `b`'nin kodu `1` olur. İkinci

karakter de b ise artık ağaçta vardır; escape gönderilmeden doğrudan 1 gönderilir. Karşı taraf b yazar; iki tarafta da b'nin sıklığı 2 olur. Ağacın değişmesi gerekir: b'nin olasılığı daha büyük olduğu için ona 1 bit verilir, escape daha uzun bir kod alır. Aynı algoritma karşıda da çalıştığı için aynı ağaç orada da oluşur. Sonra bir a gelirse önce escape, ardından a gönderilir ve iki taraf da a'yı ağacına ekler.

Bir sembol ne kadar çok gelirse ağaçta o kadar yukarıda yer almalı, yani daha az bitle kodlanmalıdır; ağaç durmadan değiştiği için kodlar da değişir. Escape kodu başlangıçta gereklidir; bütün semboller ağaçta bir kod aldıkça artık kullanılmaz. Olasılığı hep küçük olduğu için en uzun bitle ifade edilir ve belli bir aşamadan sonra bir daha karşımıza çıkmaz.

Hoca bir öğrencinin yorumuna cevaben, en sonda oluşan ağacın en çok kullanılan karakterleri en az bitle ifade ettiğini söyledi. Başlangıçta hangi karakterin çok kullanıldığı bilinmez (örneğin "kale" kelimesinde k, a, l, e birer kez geçer), ama yeni kelimeler geldikçe sistem bir doygunluğa ulaşır. Doğal bir dilde, örneğin Türkçede, hangi doküman gelirse gelsin belirli karakterler en sık geçer; a daha çok kullanıldıkça kodu kısalır, nadir bir y geldiğinde ise uzun bir kod alır.

Soru: Belli bir metin için dinamik Huffman'ın sonunda oluşan ağaç bir referans ağaç olarak, sonraki dokümanlarda kullanılamaz mı? **Cevap:** Bu durumda yine statik kodlama yapılmış olur ve başarı düşer, çünkü metnin içeriği değişir. Örneğin bir dokümanda a 30, e 25 kez geçerken başka bir dokümanda a 35, e 27 kez geçebilir; bu değerlerle ağaç kurulduğunda kodlar yine değişir. Bir dokümandan elde edilen ağaç yaklaşık bir sonuç verebilir, ama başka bir dokümanda çok daha iyi bir ağaç çıkabileceği bilinemez. Hoca seminer derslerinde, algoritmaları değiştirip küçük iyileştirmeler elde eden çalışmalar görüleceğini ekledi.

9.3 Ağacın Değişmesi Örneği

Slayttaki ağaçlar (ekrandan, 01:50:00) sıklıkları A(1), B(2), C(2), D(2), E(9) olan bir durumdan başlar. Ağaçta $1+2=3$, $2+2=4$, $3+4=7$, $7+9=16$ toplamları vardır:

```
A(1)+B(2)=3, C(2)+D(2)=4, 3+4=7, 7+E(9)=16
A geldi, A=2:  A+B=4, C+D=4, 8, 8+E=17   (ağaç değişmedi)
A geldi, A=3:  D+B=4, C+A=5, 9, 9+E=18   (A ikilerden sonraya geçti)
A geldi, A=4:  D+B=4, C+A=6, 10, 10+E=19 (ağaç değişmedi)
A geldi, A=5:  A(5) ve (C(2) + (D+B=4) = 6) -> 10, 10+E=19
```

A'nın sıklığı 2 olunca ağaç yeniden kurulsa da aynı ağaç çıkar. A 3 olunca, küçükten büyüğe yerleştirme kuralı gereği A'nın yeri değişir ve ikilerden sonraya gelir; E, B ve C'nin kodları değişmez, D ve A'nın kodları değişir. A 4 olunca yine değişiklik olmaz. A 5 olunca bir değişim olur: A'nın bulunduğu alt ağacın toplamı, A'nın kendi sıklığından küçük kalır; ağaç yeniden düzenlenirken bir alt ağaç koparılıp başka bir yere yerleşir. Her seviyede sıralamanın küçükten büyüğe olması korunur; sıra bozulduğu anda yer değiştirme yapılır.

Ağacı yeniden kurarken her seferinde bütün değerler baştan sıralanmaz; amaç yaprakların ve her hiyerarşinin küçükten büyüğe sıralı kalmasıdır. Hoca A'nın kodunun bir süre 3 bit kalıp sonra 2 bite düştüğünü, sıklıklar değiştikçe yeniden değişeceğini gösterdi.

9.4 Sayaç Taşması

Bu algoritmanın aksayan bir tarafı vardır: sıklıklar sayılır ve gittikçe artar; sayaç için bir bayt mı iki bayt mı ayrılacağı bilinemez. Bu yüzden sıklık değerinin tutulduğu yer işaretsiz 16 bit alınır; 65.535'e kadar değer tutulabilir. Herhangi bir sıklık bu sayıya ulaşırsa artık artırılamaz, çünkü 16 bite sığmaz. O zaman bütün sıklık değerleri tamsayı bölmesiyle ikiye bölünür. Ağaç değişmez, aynı kalır; sayma bu değerler üzerinden sürer. Bu kontrol yapılmazsa, örneğin 17 bitlik bir sıklık oluştuğunda değer kaybedilir; sembol çok az gelmiş gibi görünür, ağaçta en fazla bitle ifade edileceği bir yere kayar ve "işin rengi değişir".

Kod uzunluğu için de benzer bir sorun vardır. Kodlar için 16 bitlik bir alan ayrıldıysa ve ağacın seviyesi 16'yı geçerse bu alan yetmez. Örneğin slayttaki ağaçta 0, 1, 2, 3. seviyeler vardır, bu yüzden kodlar 3 bit uzunluğundadır. Çözüm olarak ya alan genişletilir (değişkenin boyu değiştirilip ağaç yeniden kurulur) ya da bağlı liste kullanılır: kodun sığmayan kısmı başka bir yerde tutulur ve oraya link verilir. Hoca bağlı listeli yolun kodlanmasının zor olduğunu söyledi.

10. Huffman Algoritmasının Farklı Bir Versiyonu

Slayttaki maddeler (ekrandan, 02:00:00):

- Basit, ancak daha az etkili bir yöntemdir.
- Eşit olasılıklı, n elemanlı değişken uzunluktaki kodlar için uygundur.
- Sembollerin olasılıkları eşit değilse etkili değildir.
- $n \times 3$ boyutunda bir tablodan oluşup, kolon değerleri sembolün kendisi, sıklığı ve kodudur.
- İkinci kolonun değeri değiştikçe satırlar yer değiştirir, ancak üçüncü kolon sabit kalır.
- Bu yöntemde count değerinde taşma oluşur; bunu çözmek için count değeri yarıya bölünür.

Hoca yöntemin, olasılıkları birbirine yakın sembollerde kullanılması gerektiğini söyledi. Kodlar önceden çıkarılmıştır (örneğin dinamik Huffman'dan çıkarılmış kodlar olabilir). Başlangıçta bütün sayaçlar 0'dır:

```
Name Count Code | Name Count Code | Name Count Code | Name Count Code
a1   0     0    | a2   1     0    | a2   1     0    | a4   2     0
a2   0     10   | a1   0     10   | a4   1     10   | a2   1     10
a3   0     110  | a3   0     110  | a3   0     110  | a3   0     110
a4   0     111  | a4   0     111  | a1   0     111  | a1   0     111
```

Bir `a2` gelir; sayacı 1 olur. En fazla kullanılan o olduğu için daha az bitle ifade edilmelidir: `a2` yukarı alınır, `a1` aşağı taşınır. Sonra bir `a4` gelir; `a2`'nin altına çıkar, `a1` onun yerine iner. Bir `a4` daha gelir; sayacı 2 olur, `a2` ile yer değiştirir. Böylece en çok gelen sembol en az bitle kodlanır. Bu yöntemde de sayaç taşarsa count değeri yarıya bölünerek devam edilir.

11. Canonical Huffman Kodlama

11.1 Neden Geliştirildi?

Hoca aradan sonra "Huffman'dan kurtulamadınız" diyerek devam etti. Huffman bir takım prefix kodlar üretir, ama üretilebilecek bütün prefix kodları üretmez; algoritma gereği arada olabilecek bazı prefix kodları atlar. Bu yüzden geniş bir alfabe için çıkarılmış Huffman kodlarıyla sıkıştırılmış dosyaların başlıkları oldukça büyük olur. Burada ağaç gönderilmez; semboller ve sembollerin kod değerleri gönderilir. Bütün prefix kodlar oluşturulmadığından bazı semboller gereğinden fazla bitle kodlanır ve başlık büyür. Canonical Huffman, Huffman algoritmasının atladığı prefix kodları da oluşturan bir dönüşümle başlığı biraz kısaltmayı amaçlar. Hoca bunun çok büyük bir kazanç sağlamadığını söyledi.

Slayttaki maddeler (ekrandan, 02:10:00):

- Huffman prefix kodları ile sıkıştırılmış dosyaların başlıklarına kıyasla, geniş alfabeli dosyalarda daha küçük dosya başlıkları oluşturmaya olanak sağlar.
- Hızlı kod dönüşüme sahiptir.
- Huffman algoritması ile oluşturulamayıp atlanan prefix kodların oluşturulmasına izin verir.

Küçük başlık alanı ve kodların hızlı geri dönüşüm özelliği nedeniyle hızlı kod çözme gerektiren uygulamalarda kullanılır. Hoca genelde ses dosyalarında ve arşiv dosyalarında ağırlıklı kullanıldığını söyledi.

11.2 Algoritma

Canonical Huffman da önce Huffman kodlarının oluşturulmasıyla başlar: elde semboller, olasılıkları ya da sıklıkları ve onlar için oluşturulmuş Huffman prefix kodları vardır. Alfabedeki her karakterin Huffman kod uzunlukları kullanılarak canonical Huffman kodlar oluşturulur:

- Prefix kodların uzunlukları büyükten küçüğe sıralanır. En uzun uzunluğa L_{max} denir.
- Bu uzunlukta k_1 tane kod varsa, L_{max} uzunluğunda, 0'dan başlayıp her seferinde 1 eklenerek k_1 tane ardışık binary sayı yazılır.
- Sonra sıradaki küçük uzunluk alınır; bu uzunlukta k_2 kod olsun. Bir önceki grubun son kodundan yeni uzunluk kadar bit alınır, buna 1 eklenir; k_2 kod tamamlanana kadar 1 eklenmeye devam edilir.

- İşlem bütün uzunluklar bitene kadar sürer.

Hoca algoritmanın karışık göründüğünü ama örnekle kolay anlaşılacağını söyledi.

11.3 Örnek

Slayttaki soru (ekrandan, 02:20:00): Huffman algoritması kullanılarak $\{a, b, c, d, e, f, g, h\}$ alfabesi için elde edilen Huffman kodlarının uzunluk değerleri sırasıyla $(5, 5, 5, 5, 3, 2, 2, 2)$ verilmiş olsun. Bu alfabe için hesaplanacak canonical Huffman kodları ne olur?

- $L_{max} = 5$, $k_1 = 4$: 5 bitlik, 0'dan başlayan dört kod: `00000`, `00001`, `00010`, `00011`.
- $L_{max-1} = 3$, $k_2 = 1$: bir önceki grubun son kodu `00011`'dir. Yeni uzunluk 3 olduğu için soldaki 3 bit `000` alınır, 1 eklenir: `001`.
- $L_{max-2} = 2$, $k_3 = 3$: son kod `001`'in soldaki 2 biti `00` alınır; $00 + 1 = 01$, $01 + 1 = 10$, $10 + 1 = 11$.

Sonuç: `a = 00000`, `b = 00001`, `c = 00010`, `d = 00011`, `e = 001`, `f = 01`, `g = 10`, `h = 11`. Hoca bunların hepsinin prefix olduğunu, örneğin hiçbirinin beş tane 0 ile başlamadığını gösterdi.

> **Ders sonrası not:** Hoca 3 bitlik adımı anlatırken önce "soldaki 3 bitini at, geri kalanını al" dedi, ardından özür dileyerek düzeltti. Slayttaki işlem, bir önceki son koddan soldaki L biti almak (yani sağdaki fazla bitleri atmak) ve 1 eklemektir: `00011` → `000` → `001`.

Header oluşturulurken Huffman kodları yerine bu canonical haller kullanılır; aradaki eksik prefix kodlar da ortaya çıkar. Ek bilgilerle canonical kodlardan özgün hallere geri dönülebilir.

> **Ek bilgi (derste anlatılmadı):** Bu örnekte kod uzunlukları Kraft eşitsizliğini eşitlikle sağlar: $4 \cdot 2^{-5} + 2^{-3} + 3 \cdot 2^{-2} = 1$. Canonical kodlarda kodlar yalnız uzunluklardan yeniden üretilebildiği için başlığa sembol başına yalnız kod uzunluğunu yazmak büyük olasılıkla yeterlidir; başlığı küçülten şey budur.
>
> **Hocanın vurgusu:** Canonical Huffman çok kullanılan bir yöntem değildir. Göstermek istenen, araştırmacıların bir algoritmayı alıp orasıyla burasıyla oynayarak yeni versiyonlar çıkarmasıdır; LZW algoritmalarında da bu görülecektir. Sonra "input şöyle bir veri olursa bu yöntem daha iyi sonuç verir" diye yayın yaparlar.

12. MNP5

12.1 Genel Yapı

Hoca 5.1 bölümünü bitirip 5.2'ye, yani istatistiksel yöntemlerin devamına geçti. Slayttaki maddeler (ekrandan, 02:26:50):

- Microcom tarafından modemler için geliştirilmiş bir sıkıştırma yöntemidir.
- MNP5 ve MNP7 diye iki versiyonu mevcuttur. En çok tercih edilen MNP5'tir.
- İki aşamalı bir yöntemdir: Run Length Encoding ve Adaptive Frequency Encoding.
- RLE: tekrar eden üç ve üçten fazla sembol varsa üçü arka arkaya yazılıp, dördüncü bayta tekrar sayısı yazılır.

MNP, Microcom Network Protocol'ün kısaltmasıdır; Microcom modemler için sıkıştırma algoritması geliştiren bir şirkettir. MNP5 tek boyutlu, MNP7 iki boyutludur. MNP5 yazması da kullanması da kolay olduğu için daha çok tercih edilir.

Hoca daha önce RLE'nin bağımsız olarak kullanıldığını hatırlattı; Huffman da bağımsız kullanılabilir. Ama bazı yöntemler başka yöntemlerin içinde bir alt adım olarak kullanılır; RLE burada MNP5 ve MNP7'nin bir adımıdır. RLE'nin farklı versiyonları olduğu için burada hangisinin kullanıldığı önemlidir: 3 tekrar varsa üç sembol yazılır ve tekrar sayısı (run değeri) yerine 0 yazılır; 4 tekrar varsa üç sembol ve run değeri 1 yazılır.

12.2 İkinci Aşama: Adaptif Frekans Kodlama

İkinci aşama birinci aşamanın sonuçlarını kullanır ve Huffman kodunun bir versiyonu olarak anlatılan yönteme benzer (Bölüm 10). Yani ikinci aşamaya geçmeden önce RLE değerleri çıkarılır: semboller ve run değerleri. Slayttaki maddeler (ekrandan, 02:30:00):

- 256 satır ve 2 sütunluk bir tablo ile başlar.
- Her bir satır 8 bitlik 0-255 arası sayıya karşılık gelir.
- Birinci kolon frekans değeridir ve başlangıç değeri 0'dır.
- İkinci kolon "token" adı verilen değişken uzunluktaki koddan oluşur.
- Her bir token 3 bitlik header (başlık) bilgisi ile başlar ve bunları kod bilgisi izler.
- Üç farklı durum dışında, 2 tane 1 bitlik (0-1), 4 tane 2 bitlik (0-3), 8 tane 3 bitlik (0-7), 16 tane 4 bitlik (0-15), 32 tane 5 bitlik (0-31), 64 tane 6 bitlik (0-63) ve 128 tane 7 bitlik (0-127) adet koddan oluşur.
- Bahsedilen 3 farklı kodun ilk iki kodu (`000|0`, `000|1`) ve son kodu da (`111|11111110`)'dır.

Slayttaki tablonun bir kısmı (ekrandan, 02:30:00):

```
Byte Freq Token      | Byte Freq Token      | Byte Freq Token
0    0    000|0      | 9    0    011|001    | 32   0    101|00000
1    0    000|1      | 10   0    011|010    | 33   0    101|00001
2    0    001|0      | 11   0    011|011    | 34   0    101|00010
3    0    001|1      | 12   0    011|100    | ...
4    0    010|00     | 13   0    011|101    | 247  0    111|1110111
5    0    010|01     | 14   0    011|110    | ...
6    0    010|10     | 15   0    011|111    | 254  0    111|1111110
7    0    010|11     | 16   0    111|0000   | 255  0    111|11111110
8    0    011|000    | 17   0    111|0001   |
(18-25 ve 35-246 aynı düzende devam eder)
```

Hoca header'ın solundaki 3 bitlik değerin, çizginin sağında kaç bit yazılacağını gösterdiğini anlattı: header 1 ise 1 bitle iki kod (0 ve 1), 2 ise 2 bitle dört kod (`00, 01, 10, 11`), 3 ise 3 bitle sekiz kod yazılır; 4 bitle 16, 5 bitle 32, 6 bitle 64, 7 bitle 128 kod. İlk ikisi farklıdır: tablo 0 ile başlatıldığı için ve 0 bitle bir şey yazılamayacağı için, header'ları farklı olduğundan prefix özelliğini bozmayacak biçimde `000|0` ve `000|1` yazılır. Son kodda ise 255'ten sonra başka kod gelmeyeceği için fazla bit oluşmasın diye bir bit düzenlemesi yapılır.

> **Ders sonrası not:** Slayttaki tabloda 16-31 baytlarının header'ı `111` yazılmış (`111|0000` … `111|1111`). Hoca "şurası 4 olacak, bu bir yerden alınmış" diyerek burasının hatalı olduğunu söyledi. Header'lar gruplar boyunca 0, 1, 2, 3 diye arttığı için 4 bitlik kodların (16-31) header'ı büyük olasılıkla `100`, 6 bitlik kodlarınki `110` olmalıdır; aksi halde 111 ile başlayan 7 bitlik kodlarla çakışır.

12.3 Tablonun Güncellenmesi

Slayttaki ikinci adım (ekrandan, 02:39:50):

- Tablodaki frekans kolonu 0 ile başlar. Input stream'den okunan ilk bayta karşılık gelen token tablodan çıkış stream'e yazılır ve frekans değeri bir artırılır.
- Frekans değeri büyük olan bayt, kendi üzerinde yer alan daha kısa token ile gösterilebilmesi için token'ların yeri değiştirilir (sadece token'lar).
- Frekans değeri 8 bitlik bir alanda tutulur ve her değişiklikte maksimum değere ulaşılıp ulaşılmadığı kontrol edilir. Eğer ulaşılmışsa tüm sıklık değerleri ikiye bölünür.

Hoca bunu şöyle anlattı: yeni bir sembol gelir, diyelim tabloda 30. sıradadır. 30'un frekansı 1 yapılır. Artık en fazla kullanılan sembol odur; kendi bitiyle değil, kendi üzerindeki en kısa token ile gösterilmelidir. Frekansı üsttekilerle karşılaştırılır; hepsinden büyük olduğu için en üstteki (en kısa) token'ı gösterir, diğerleri birer aşağı kayar. Sonra bir başka sembol (örneğin M) gelir, frekansı 1 olur; frekansı 0 olan satırları geçerek ikinci sıraya kadar çıkar, çünkü birinci sıradakinin de frekansı 1'dir. Bir başka sembol gelirse o da frekansı 0 olanların üzerine çıkar.

Tablonun güncellenmesi iki şekilde yapılır (ekrandan, 02:34:00): her sıklık değeri artırıldığında bütün tablo yeniden sıralanır ya da tablo içerisinde pointer kullanılarak sıralama gerçekleştirilir. Slayttaki şekilde F (frekans) ve P (pointer) sütunları ile Q (pointer) ve C (kod) sütunları vardır; frekansı artan sembolün pointer'ı daha kısa kodu gösterecek biçimde değiştirilir. Hoca örnekte 8'e karşılık gelen sembolün frekansı 1 olunca 6 bitlik yerine yukarıdaki 4 bitlik token ile gösterileceğini, pointer'ların birer aşağı kayarak düzenlendiğini anlattı.

12.4 İki Aşamanın Birleştirilmesi

Slayttaki örnek (ekrandan, 02:37:30): `52 52 52 6`.

- Bu, RLE çıktısıdır: üç tane 52 yazılmış, ardından tekrar sayısı 6 gelmiştir; toplam 9 tane 52 vardır.
- Tabloda 53. sıradaki token değeri stream'e yazılır (tablo 0'dan başladığı için 52 değeri 53. satırdadır).
- Her bayt okunduktan sonra tabloda yer değiştirme olabilir ve 52 değeri farklı token değerlerine sahip olabilir. Yani birinci 52 için o anki token yazılır, frekansı bir artar, pointer değişebilir; ikinci 52 için o anki token yazılır, frekans tekrar artar; üçüncüde de aynısı olur.
- 6 değeri için de tabloda 7. sırada yer alan token değeri yazılır, fakat frekans değeri artırılmaz; çünkü 6 bir run (tekrar sayısı) değeridir, bir sembol değildir.

Hoca bir örneği daha tahtada çözdü (ekrandan, 02:39:50). Sekizinci sembol için token `011|000` yazılır ve frekansı 1 olur; artık 8 en üstteki token'ı gösterir. Bir sonraki 8 geldiğinde yazılacak token `000|0`, yani dört tane 0'dır. Ardından tekrar sayısı 5 gelir: 5. sembolün token'ı `010|01` yazılır, run değeri olduğu için frekansı değiştirilmez.

Hoca bir örnek daha verdi: girdide `7 7 7` ve ardından tekrar sayısı 6 gelirse, birinci 7 için 7. sıradaki token alınır, frekansı bir artırılır, pointer değişir; ikinci 7 için farklı bir token yazılır; üçüncü 7 de aynı biçimde işlenir. Ardından 6 run değeri olarak yazılır ve frekansı artırılmaz.

12.5 Çıkış ve Bitiş

Slayttaki maddeler (ekrandan, 02:37:30):

- Çıkış değerinde yer alan token'lar 4 ile 11 bit arasında değer alabilir. Bu çıktılar 8 bitlik paketler halinde çıkış stream'e yazılır.
- En sona 11 tane birden oluşan "flush token" adı verilen kod eklenir (gerekirse 8'e tamamlamak için 1 eklenir) ve output stream sonlandırılır.
- MNP5'in iyi sonuç vermesi tamamıyla orijinal verinin içeriğine bağlıdır.

Elindeki token'larda hiç 11 tane 1 olmadığı için açıcı 11 tane 1 görünce işlemin bittiğini anlar. Output stream'e bayt bayt yazıldığı için son grupta 8 yerine örneğin 4 bit kalabilir; bunlar 1 eklenerek (padding) 8'e tamamlanır.

13. MNP7

MNP7, MNP5'ten daha karmaşık bir algoritmaya sahiptir ve bu yüzden modemlerde fazla tercih edilmez. Bunda da RLE ve yine Huffman'ın bir versiyonu olan adaptif bir kodlama vardır, fakat yöntem iki boyutludur. İlk bölüm yine RLE ile başlar: üç tekrardan sonra run değeri konur. İkinci bölüm Huffman benzeri kodlamadır, ama birinci dereceden Markov modeline dayanır.

Hoca birinci dereceden Markov modelini, bir karakterin olasılığının ondan önceki karaktere bakılarak hesaplanması diye hatırlattı. Örneğin işlenen karakter B ise ve önünde A varsa, token değeri ikisine birlikte bakılarak verilir; iki boyutlu olmasının sebebi budur. MNP5'te tablo 256 satırlıydı; burada önceki karakter ve şimdiki karakter için 256'ya 256'lık bir yapı vardır.

Slayttaki maddeler (ekrandan, 02:44:30):

- 256 adet, 256 x 2 boyutunda tablodan oluşur.
- Her satır 0 ile 255 arasındaki karakteri temsil eder.
- 256x2 boyutundaki her tablonun da ilk sütunu 0-255 arasındaki sayılardan ve ikinci sütunu da kullanım sıklıklarından oluşur ki başlangıç değeri 0'dır.
- Kodlar ayrı bir tabloda tutulur ve asla yer değiştirme yapılmaz.

Şekilde sütunlar şimdiki karakter (current character), satırlar önceki karakter (preceding character) olarak gösterilmiştir. Hoca örneği şöyle anlattı: `AB` dizisinde B işleniyor; B diyelim 61, A 41 olsun. 61. kolona ve 41. satıra gelinir; oradaki değer bir kod tablosundaki yere karşılık gelir ve kod buradan okunur. Frekans değerleri bu tabloda değişir; kodlar ayrı tabloda tutulur ve orada değişiklik olmaz, pointer'lar sıklık tablosunda değişir. Hoca bunu kodlamanın gerçekten karmaşık olduğunu ve modemlerde genelde MNP5'in kullanıldığını söyledi.

Örnek Sorular

Soru 1 (ekrandan, 00:20:00)

Olasılıkları $0{,}4; 0{,}2; 0{,}2; 0{,}1; 0{,}1$ olan beş sembol için iki farklı Huffman ağacı kurunuz, kodları ve ortalama mesaj uzunluklarını bulunuz.

Çözüm (derste verildi): Toplam eşit değerlerin önüne konursa kodlar `0, 10, 111, 1101, 1100`, arkasına konursa `11, 01, 00, 101, 100` olur. İkisinde de $S = 2{,}2$ bit/sembol ve entropi aynıdır.

Soru 2 (ekrandan, 00:30:00)

Soru 1'deki iki ağaç için kod uzunluklarının varyansını hesaplayınız. Hangi kod daha iyidir?

Çözüm (derste verildi): $\sigma_1^2 = 1{,}36$, $\sigma_2^2 = 0{,}16$. Veri dosyaya yazılacaksa fark yoktur; iletişim hattından gönderilecekse (özellikle dinamik Huffman'da) küçük varyanslı ikinci kod seçilir, çünkü daha küçük bir buffer yeter.

Soru 3 (derste sözlü, 00:27:48)

Olasılığı 0,4 olan bir sembol ortalama kaç bitle kodlanabilir?

Çözüm (derste verildi): $\lceil -\log_2 0{,}4 \rceil = \lceil 1{,}32 \rceil = 2$ bit; sembol 2 ya da 1 bitle kodlanır.

Soru 4 (ekrandan, 00:33:30)

Elimizde `a, e, k, l, m` ve `z` sembollerinden sırasıyla 25, 14, 8, 7, 4 ve 2 adet olsun. Her bir sembol için Huffman kod değerini bulunuz.

Çözüm (derste verildi): Genişletilmiş dizi `2 4 -6 7 8 -13 14 -21 25 -35 -60`. Kodlar: a=`1`, e=`01`, k=`001`, l=`0000`, m=`00010`, z=`00011`.

> **Çözüm (derste verilmedi):** Toplam bit $25 \cdot 1 + 14 \cdot 2 + 8 \cdot 3 + 7 \cdot 4 + 4 \cdot 5 + 2 \cdot 5 = 135$, ortalama $135/60 = 2{,}25$ bit/sembol. Entropi $H \approx 2{,}19$ bit, artıklık $R \approx 0{,}06$ bit (python ile hesaplandı).

Soru 5 (ekrandan, 01:00:00)

Soru 4'teki ağacı başlıkta göndermek için Left-Node-Right notasyonuyla okuyunuz.

Çözüm (derste verildi): Bit dizisi `0000101111`, yaprak sırası `l m z k e a`. Header'a okunacak bit sayısı (10), bu bit dizisi ve semboller yazılır.

Soru 6 (derste sözlü, 00:55:39)

Kökün sol alt ağacının solunda A, sağında B ve C yaprakları olan bir alt ağaç; kökün sağ alt ağacının solunda E, sağında F ve G yaprakları olan bir alt ağaç bulunan Huffman ağacını Left-Node-Right notasyonuyla okuyunuz.

Çözüm (derste verildi): Hoca sırasıyla `0`, `0` → A, `1`, `0` → B, `1` → C, `1`, `0` → E, `1`, `0` → F, `1` → G diye okudu. Bit dizisi bu sırayla `0010110101`, yaprak sırası `A B C E F G` olur (dizi tek parça olarak derste yazılmadı; ara adım eklendi).

Soru 7 (ekrandan, 01:33:45)

Genişletilmiş dizi ve indis tablosunu kullanarak frekansı 7 ve 2 olan sembollerin kodlarını bulunuz.

Çözüm (derste verildi): İndis 15: `0`, 7; `0`, 3; `0`, 1; `0`, 0 → `0000`. İndis 34: `1`, 16; `1`, 7; `0`, 3; `0`, 1; `0`, 0 → bitler sağdan sola yazılınca `00011`.

Soru 8 (ekrandan, 02:20:00)

$\{a, b, c, d, e, f, g, h\}$ alfabesi için elde edilen Huffman kodlarının uzunluk değerleri sırasıyla $(5, 5, 5, 5, 3, 2, 2, 2)$ verilmiş olsun. Bu alfabe için hesaplanacak canonical Huffman kodları ne olur?

Çözüm (derste verildi): `00000, 00001, 00010, 00011, 001, 01, 10, 11`.

Soru 9 (derste sözlü, 01:46:17)

Dinamik Huffman ile bir dokümanda elde edilen son ağaç, sonraki dokümanlar için referans olarak kullanılabilir mi?

Çözüm (derste verildi): Kullanılırsa statik kodlama yapılmış olur ve başarı düşer; yeni dokümanda sıklıklar farklı olacağı için en iyi kodlar da değişir. Yaklaşık bir sonuç verebilir.

Dersin Toparlaması

Bu derste Huffman kodlama baştan sona işlendi. Aynı olasılıklarla iki farklı ağaç kurulabildiği, bunların entropi, ortalama uzunluk ve artıklık bakımından aynı olduğu, ama kod uzunluklarının varyansının farklı olduğu görüldü; varyans yalnız iletim hattında, buffer ihtiyacını belirlediği için önemlidir. Sıklık değerleriyle genişletilmiş dizi ($2n - 1$ eleman, ara toplamlar negatif) kurulup ağaç çıkarıldı; kodların kökten yaprağa okunması gerektiği, ağacın header'da Left-Node-Right dal dizisiyle gönderilebileceği ve $2i + 1$ / $2i + 2$ indis kuralıyla kodların ağaç çizmeden yalnız iki diziden üretilebileceği gösterildi.

Ardından Huffman'ın statik, yarı dinamik ve dinamik türleri karşılaştırıldı; dinamik Huffman'da escape kodu, ağacın senkron güncellenmesi ve sayaç taşmasında sıklıkların yarıya bölünmesi anlatıldı. Huffman'ın tablo tabanlı basit bir versiyonu, başlığı küçültmek için kod uzunluklarından yeni kodlar üreten canonical Huffman, modemler için geliştirilen ve RLE ile adaptif token tablosunu birleştiren MNP5 ve birinci dereceden Markov modeline dayanan iki boyutlu MNP7 işlendi. Hoca bir sonraki derste faks makinelerinde kullanılan bir ve iki boyutlu sıkıştırma yöntemlerine geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.